Universidade do Minho

Mestrado em Cibersegurança

Laboratório de Cibersegurança 1

Carlos Daniel Silva Fernandes
(PG59783)

Pedro Augusto Ennes de Martino Camargo
(PG59791)

Luís Filipe Pinheiro Silva
(PG59790)

20 de abril de 2026

Conteúdo

1 Executive Summary **3**

2 Scope and Methodology **4**
2.1 Pre engagement 4

3 Phase A Findings **5**
3.1 SQL Injection in API Endpoint 5
3.1.1 Root Cause 5
3.1.2 Impact Analysis 5
3.1.3 Evidence 5
3.1.4 Remediation Recommendations 6
3.2 Broken Access Control 7
3.2.1 Root Cause 7
3.2.2 Impact Analysis 7
3.2.3 Evidence 7
3.2.4 Remediation Recommendations 8
3.3 2FA Bypass via TOTP Secret Exposure 9
3.3.1 Root Cause 9
3.3.2 Impact Analysis 9
3.3.3 Evidence 9
3.3.4 Remediation Recommendations 10

4 Phase B Findings **11**

5 Cross-Phase Analysis **12**

6 Conclusion **13**

7 Appendix A - Evidence **14**
7.1 SQL Injection in `/rest/products/search` 14

8 Appendix B - Structured Event Log **15**

Capítulo 1

Executive Summary

High-level summary of scope, key findings, and top recommendations. Written for a non-technical audience. 1 page maximum.

Capítulo 2

Scope and Methodology

Following the PTES framework, this section details the engagement scope, rules of engagement, intelligence gathering, and attack planning processes. It also outlines the tools and techniques used, and any deviations from the standard PTES approach.

2.1 Pre engagement

The applications targeted in this engagement were the **OWASP Juice Shop** and the **Damn Vulnerable Web Application (DVWA)**. Both applications are intentionally vulnerable web applications designed for security training and testing purposes.

In this report, we will document the findings from our engagements with both applications, following the PTES and WSTG frameworks. The report is structured to provide a comprehensive overview of our activities, findings, and recommendations for each phase of the engagement. However, we will not exploit and destroy any systems.

We plan to use tools such as **Burp Suite** for web application testing, **Nmap** for network scanning, and **Metasploit** for exploitation. We will also utilize manual testing techniques to identify vulnerabilities and assess the security posture of the applications.

Furthermore, is important to note that we will adhere to ethical guidelines and legal requirements throughout the engagement, ensuring that all activities are conducted in a responsible and professional manner.

Capítulo 3

Phase A Findings

3.1 SQL Injection in API Endpoint

Vulnerability Overview

Vulnerability Class	Injection (SQL Injection)
Location	API Endpoint: `/rest/products/search`
CVSS v4 Score	9.1 (Critical)
MITRE ATT&CK	Tactic: Initial Access Technique ID: T1190

3.1.1 Root Cause

The API endpoint `/rest/products/search` constructs SQL queries dynamically using user-supplied input from URL parameters. This is done through string concatenation without proper sanitization or validation.

As a result, an attacker can inject malicious SQL payloads, manipulate query execution, and retrieve or modify unintended data from the SQLite database.

3.1.2 Impact Analysis

This vulnerability critically impacts all three pillars of the CIA triad:

Security Property	Impact
Confidentiality	Unauthorized access to sensitive data via `SELECT` queries.
Integrity	Data manipulation or corruption through `INSERT`, `UPDATE`, or `DELETE`.
Availability	Malformed queries or exploitation can trigger database errors, leading to service disruption.

3.1.3 Evidence

The vulnerability was confirmed using SQL injection payloads submitted through URL parameters.

```
.../rest/products/search?q=apple')) UNION SELECT id,email,password,4,5,6,7,8,9 FROM users --
```

Listing 3.1: URL Payload

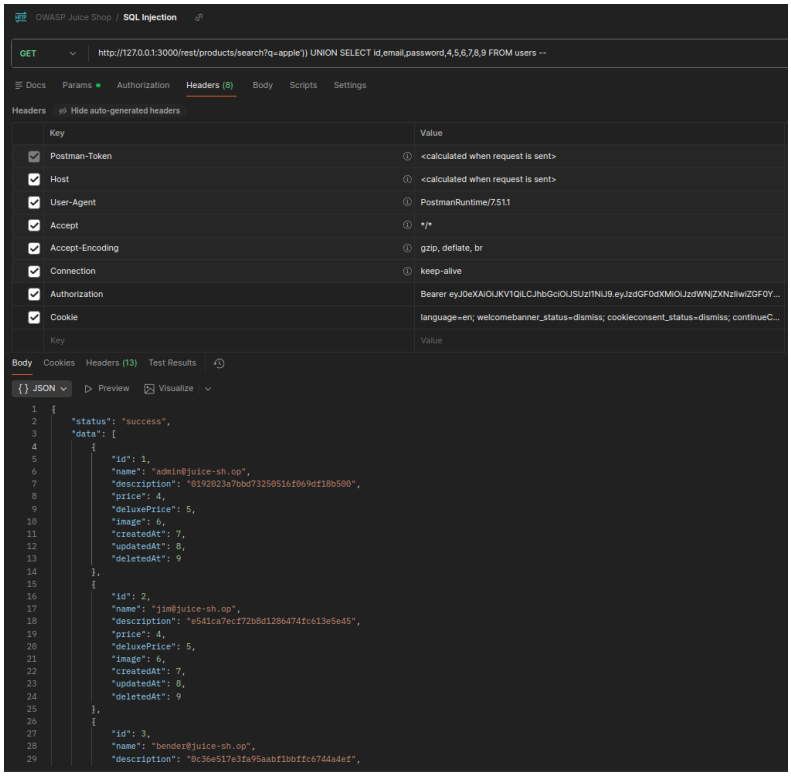

Figura 3.1: SQL injection payload submitted to the `/rest/products/search` endpoint via Postman. The response includes all users names and passwords, demonstrating successful exploitation.

3.1.4 Remediation Recommendations

Recommended Fixes

1. Use Parameterized Queries
Replace dynamic SQL construction with prepared statements or parameterized queries to ensure user input is treated strictly as data.

2. Input Validation
Implement strict validation and sanitization of all user inputs, especially those used in database queries.

3. Secure Error Handling
Avoid exposing internal database errors. Replace verbose messages such as:

```
"error": "syntax error near UNION SELECT..."
```

with generic responses:

```
"error": "Invalid request"
```

4. Principle of Least Privilege
Ensure the database user has minimal required permissions to limit the impact of exploitation.

3.2 Broken Access Control

Vulnerability Overview

Vulnerability Class	Broken Access Control
Location	API Endpoint: `/rest/basket/[Basket-ID]`
CVSS v4 Score	5.3 (Medium)
MITRE ATT&CK	Tactic: Initial Access
	Technique ID: T1078

3.2.1 Root Cause

The server does not validate cookie session and JWT token with userId. If a malicious user connected use the api route to get another basket by changing the ID in the URL, access to data he's not supposed to have is granted.

With this, an attacker has access to other users' baskets by manipulating the Basket-ID in the URL, without proper authorization checks, returning non-sensitive business related information such as **date of creation**, **total price**, and **product details**.

3.2.2 Impact Analysis

This vulnerability primarily impacts confidentiality, as it allows unauthorized access to other users' basket information. While the data exposed may not be highly sensitive, it can still lead to privacy concerns and potential misuse of information.

Security Property	Impact
Confidentiality	Unauthorized access to other users' basket information, including product details and total price.
Integrity	No direct impact on data integrity, as the vulnerability does not allow modification of data.
Availability	No direct impact on availability, as the vulnerability does not cause service disruption.

3.2.3 Evidence

The vulnerability was confirmed by manipulating the Basket-ID in the URL and successfully retrieving another user's basket information without proper authorization.

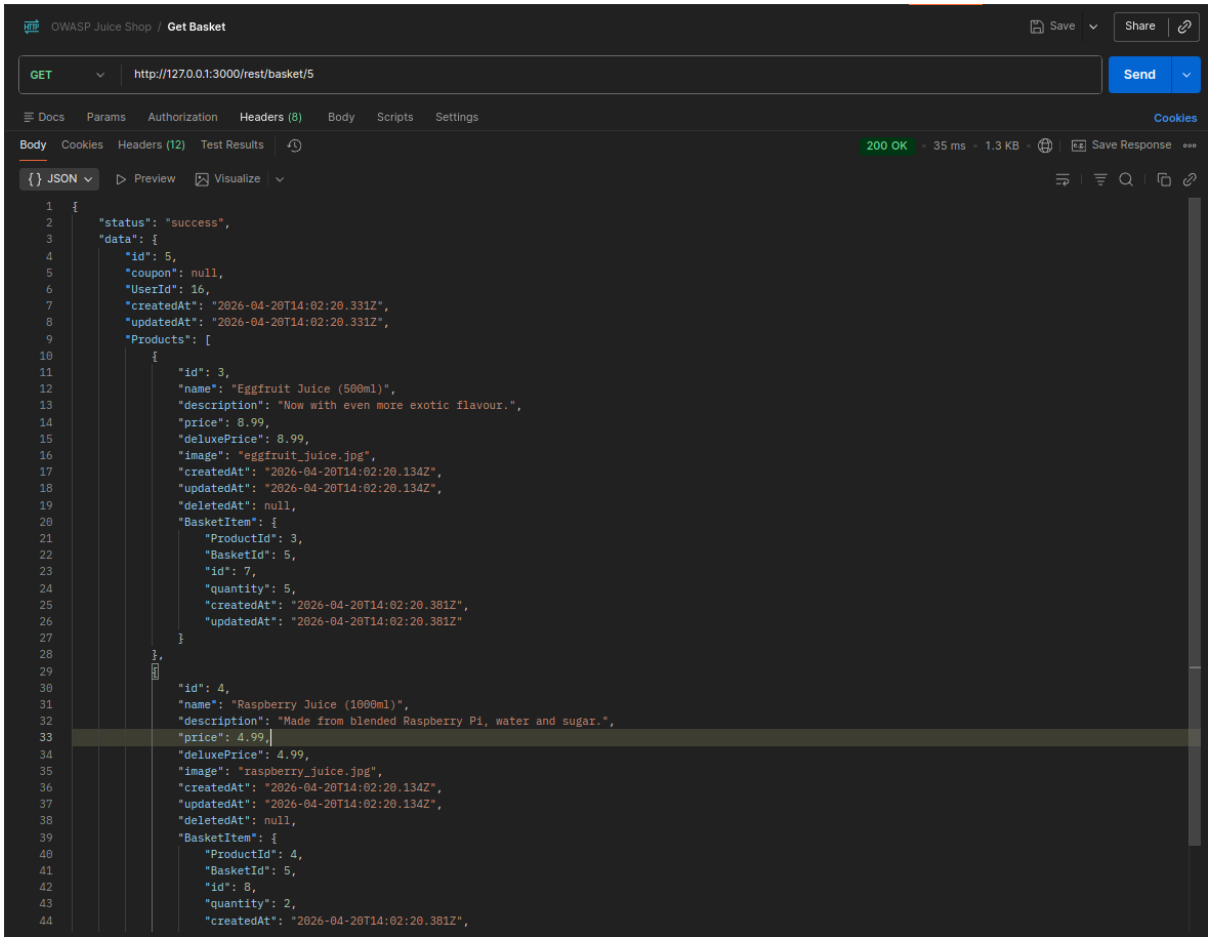

Figura 3.2: Postman request demonstrating broken access control by changing the Basket-ID in the URL to access another user's basket information. (Current user basket id is 6)

3.2.4 Remediation Recommendations

Recommended Fixes

1. Implement Proper Authorization Checks
Ensure that the server validates the user's session and JWT token against the requested Basket-ID to confirm that the user has permission to access that specific resource.

2. Enforce Role-Based Access Control (RBAC)
Implement RBAC to restrict access to resources based on user roles and permissions, ensuring that users can only access their own data.

3. Use Unique Identifiers
Consider using unique identifiers that are not easily guessable for resources like baskets, to make it harder for attackers to enumerate and access other users' data.

3.3 2FA Bypass via TOTP Secret Exposure

Vulnerability Overview

Vulnerability Class	Authentication
Location	Database and Login Panel
CVSS v4 Score	8.9 (High)
MITRE ATT&CK	Tactic: Credential Access
	Technique ID: T1555 (Credentials from Password Stores)

3.3.1 Root Cause

Due to the presence of other vulnerabilities in the system, it is possible to extract sensitive user data directly from the database. Among this data is the TOTP secret used for two-factor authentication (2FA), which is stored in plaintext without any form of encryption or protection.

Because the secret is not encrypted, an attacker can easily import it into a standard 2FA application (e.g., Google Authenticator) and generate valid one-time passwords for the victim account.

3.3.2 Impact Analysis

This vulnerability compromises authentication mechanisms and allows full account takeover:

Security Property	Impact
Confidentiality	Exposure of TOTP secrets enables attackers to impersonate users.
Integrity	Unauthorized access allows attackers to modify user data and system state.
Availability	Accounts may be locked or abused, impacting legitimate user access.

3.3.3 Evidence

The TOTP secret was retrieved from the database in plaintext format. The field storing the secret was not protected or encrypted, allowing direct reuse in a 2FA application.

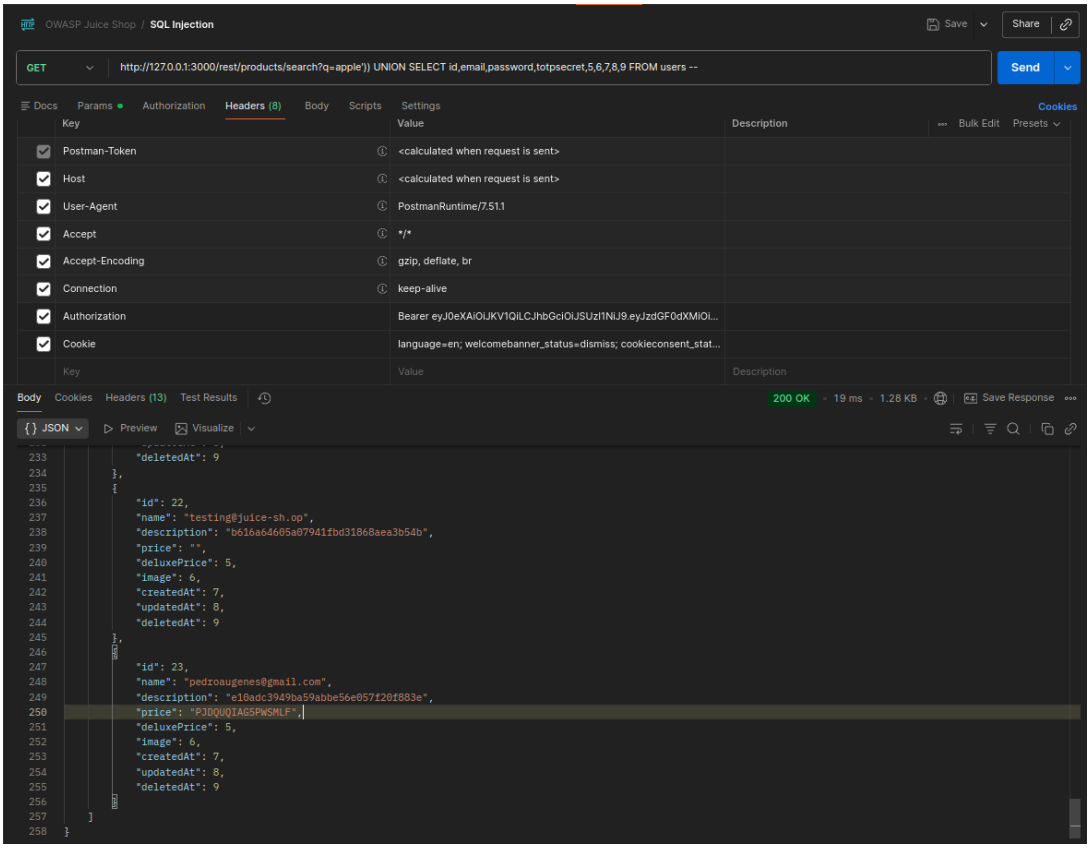

Figura 3.3: Postman request demonstrating the leakage of the TOTP secret from the database if a user has 2FA enabled. "price"field contains the TOTP secret in plaintext, which can be imported into a 2FA app to generate valid one-time passwords.

3.3.4 Remediation Recommendations

Recommended Fixes

1. Encrypt TOTP Secrets
Store TOTP secrets securely using strong encryption (e.g., AES with proper key management). Secrets must never be stored in plaintext.

Capítulo 4

Phase B Findings

Findings from the DVWA engagement, each documented to the standard in §5 above. Includes the Phase B attack plan (§6.1).

Capítulo 5

Cross-Phase Analysis

Technique transfer, adaptation, and reflection on DVWA's medium-security controls, as specified in §6.3 above.

Capítulo 6

Conclusion

Prioritised remediation recommendations for both targets. Overall assessment of engagement quality and lessons learned

Capítulo 7

Appendix A - Evidence

7.1 SQL Injection in `/rest/products/search`

This appendix section contains the screenshots and supporting material referenced in Section **??**.

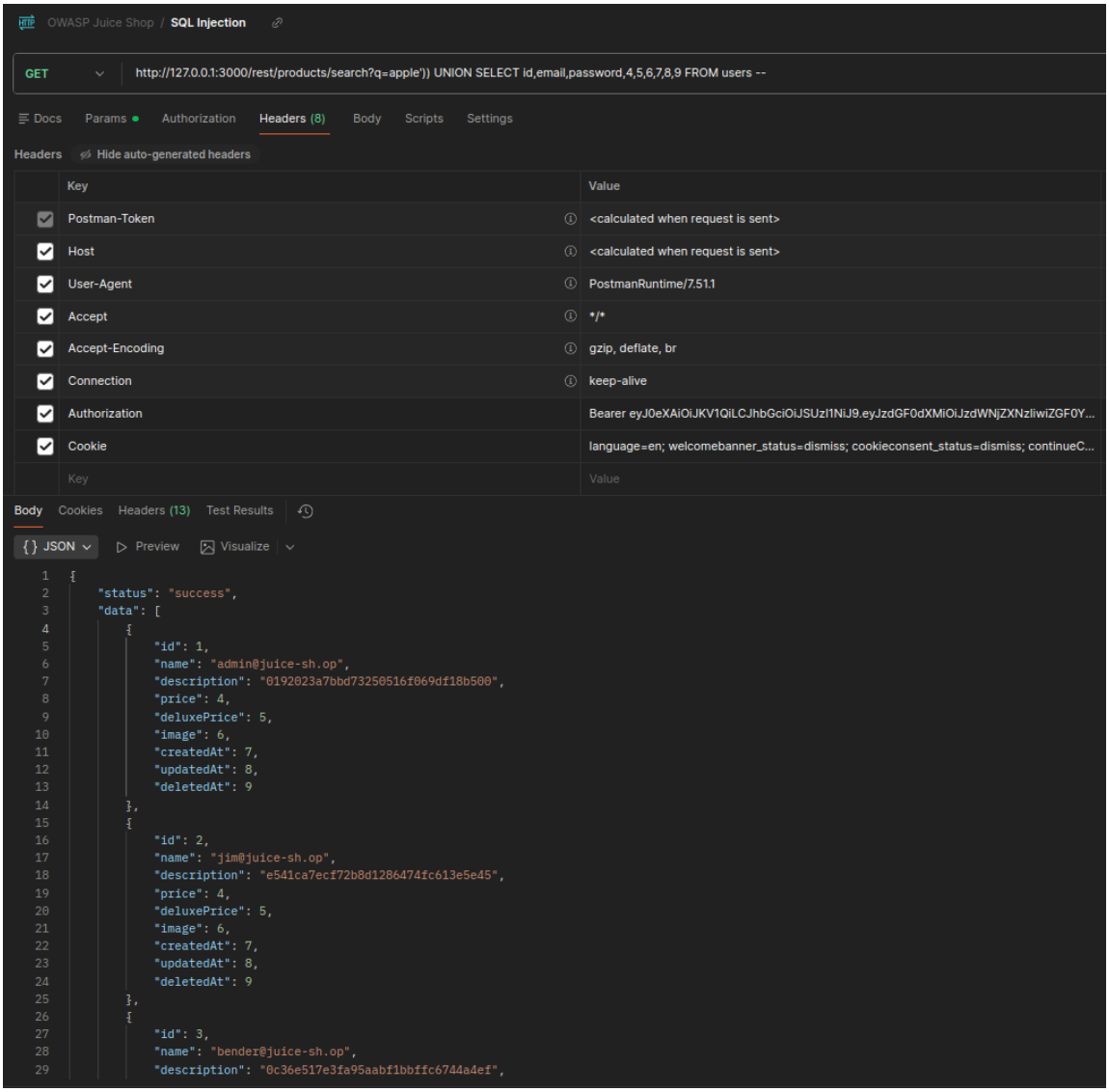

Figura 7.1: SQL injection payload submitted to the `/rest/products/search` endpoint.

Capítulo 8

Appendix B - Structured Event Log

Complete event log of Phase B exploitation activity, in the format specified by the instructor